

04 — PyTorch & Deep Learning

Time: ~5-6 hours | **Level:** Intermediate

What you'll learn:

- PyTorch tensors and autograd (automatic differentiation)
- Building neural networks with `nn.Module`
- Training loops, optimisers, and learning rate schedulers
- Mini-batch gradient descent
- Regularisation: Dropout, Batch Norm, Weight Decay
- Saving & loading models

Prerequisites: Notebook 03 (neural networks from scratch)

Why PyTorch?

In Notebook 03, we built backpropagation by hand. In production, nobody does that. PyTorch gives us:

- **Autograd:** automatic gradient computation for ANY computation graph
- **GPU acceleration:** 10-100x faster training
- **nn.Module:** clean abstractions for building models
- **Ecosystem:** HuggingFace, torchvision, torchaudio all built on PyTorch

```
In [ ]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
import matplotlib.pyplot as plt

print(f'PyTorch version: {torch.__version__}')
print(f'CUDA available: {torch.cuda.is_available()}')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')
```

1. Tensors — The Foundation

A **tensor** is a multi-dimensional array (like NumPy's ndarray) but with two superpowers:

1. Can live on GPU for parallel computation
2. Tracks operations for automatic gradient computation

```
In [ ]: # — Creating Tensors —————

# From Python lists
t1 = torch.tensor([1.0, 2.0, 3.0])
print(f'From list: {t1}, dtype: {t1.dtype}')

# From NumPy (zero-copy when possible)
arr = np.array([[1, 2], [3, 4]], dtype=np.float32)
t2 = torch.from_numpy(arr)
print(f'From NumPy: {t2}')

# Common initialisations
zeros = torch.zeros(3, 4)           # 3x4 matrix of zeros
ones = torch.ones(2, 3)             # 2x3 matrix of ones
rand = torch.randn(2, 3)            # 2x3 matrix of random normal
eye = torch.eye(3)                  # 3x3 identity matrix

# Shape operations (same as NumPy)
t3 = torch.randn(2, 3, 4)
print(f'\nShape: {t3.shape}')
print(f'Reshape: {t3.reshape(6, 4).shape}')
print(f'Transpose: {t3.permute(0, 2, 1).shape}') # swap dims 1,2
print(f'Unsqueeze: {t3.unsqueeze(0).shape}')     # add batch dim
```

2. Autograd — Automatic Differentiation

This is what makes PyTorch magical. Set `requires_grad=True` on a tensor, and PyTorch will track every operation. Then call `.backward()` to compute all gradients automatically.

Remember from Notebook 03: we manually coded `backward()` with chain rule. PyTorch does this for ANY computation graph — including ones with billions of params.

```
In [ ]: # — Autograd in action —————

# Simple example:  $f(x) = x^2 + 3x + 1$ , find  $df/dx$  at  $x=2$ 
x = torch.tensor(2.0, requires_grad=True) # ← tells PyTorch to track
f = x**2 + 3*x + 1                         #  $f(2) = 4 + 6 + 1 = 11$ 
f.backward()                              # compute  $df/dx$ 
print(f'f(2) = {f.item()}')
print(f'df/dx at x=2: {x.grad.item()}')   #  $df/dx = 2x + 3 = 7$ 
print(f'Manual check:  $2*2 + 3 = {2*2 + 3}$ ')

print('\n— Now with matrices —')
# This is what happens inside a neural network
W = torch.randn(3, 2, requires_grad=True)
x_in = torch.randn(5, 3) # 5 samples, 3 features
y_true = torch.randn(5, 2)

# Forward pass
y_pred = x_in @ W           # matrix multiply
loss = ((y_pred - y_true)**2).mean() # MSE loss
```

```
# Backward pass – ONE line instead of 50 lines of manual backprop
loss.backward()

print(f'Loss: {loss.item():.4f}')
print(f'W.grad shape: {W.grad.shape}') # gradient has same shape as W
print(f'W.grad:\n{W.grad}')
print('\n💡 PyTorch computed all gradients automatically!')
```

3. nn.Module — Building Networks the PyTorch Way

Instead of manually managing weight tensors, PyTorch provides `nn.Module` :

- Define layers in `__init__`
- Define forward pass in `forward`
- Weights, gradients, GPU transfer all handled automatically

```
In [ ]: # — Our Notebook 03 network, now in PyTorch —————

class SimpleNet(nn.Module):
    """Same 2-layer network from Notebook 03, but in 5 lines."""

    def __init__(self, n_input, n_hidden, n_output):
        super().__init__()
        self.layer1 = nn.Linear(n_input, n_hidden) # W1, b1
        self.layer2 = nn.Linear(n_hidden, n_output) # W2, b2
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.layer1(x)) # z1 → ReLU(z1)
        x = self.sigmoid(self.layer2(x)) # z2 → sigmoid(z2)
        return x

model = SimpleNet(4, 8, 1)
print(model)
print(f'\nTotal parameters: {sum(p.numel() for p in model.parameters())}')
print(f'\n💡 Compare: In Notebook 03 this was ~60 lines. PyTorch: ~10 lines.')
print('    AND we get autograd, GPU support, and serialisation for free.')
```

```
In [ ]: # — Generate the same dataset from Notebook 03 —————

np.random.seed(42)
n_samples = 500
X_np = np.random.randn(n_samples, 4).astype(np.float32)
y_np = ((X_np[:, 0] + X_np[:, 1] + 0.3 * X_np[:, 2]) > 0.5).astype(np.float32)

# Convert to tensors
X_train = torch.from_numpy(X_np[:400])
y_train = torch.from_numpy(y_np[:400])
X_test = torch.from_numpy(X_np[400:])
y_test = torch.from_numpy(y_np[400:])

# DataLoader: handles batching, shuffling automatically
```

```

train_ds = TensorDataset(X_train, y_train)
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)

print(f'Training: {len(X_train)} samples')
print(f'Testing: {len(X_test)} samples')
print(f'Batches per epoch: {len(train_loader)} (batch_size=32)')

```

4. The Training Loop

PyTorch's training loop has a very consistent pattern:

```

for epoch in range(n_epochs):
    for batch_x, batch_y in dataloader:
        # 1. Forward pass
        predictions = model(batch_x)
        loss = loss_fn(predictions, batch_y)

        # 2. Backward pass
        optimizer.zero_grad() # clear old gradients
        loss.backward()        # compute new gradients
        optimizer.step()       # update weights

```

You'll see this pattern in EVERY PyTorch project, from simple classifiers to LLM training.

```

In [ ]: # — Full Training Loop —————

torch.manual_seed(42)
model = SimpleNet(4, 8, 1)
loss_fn = nn.BCELoss() # Binary Cross-Entropy
optimizer = optim.Adam(model.parameters(), lr=0.01) # Adam > SGD in practice

# tracking
train_losses = []

for epoch in range(200):
    epoch_loss = 0
    for batch_x, batch_y in train_loader:
        # Forward
        y_pred = model(batch_x)
        loss = loss_fn(y_pred, batch_y)

        # Backward
        optimizer.zero_grad() # NEVER forget this!
        loss.backward()
        optimizer.step()

    epoch_loss += loss.item()

    avg_loss = epoch_loss / len(train_loader)
    train_losses.append(avg_loss)

    if epoch % 50 == 0:
        with torch.no_grad(): # disable gradient tracking for eval
            test_pred = model(X_test)

```

```

        test_acc = ((test_pred > 0.5) == y_test).float().mean()
        print(f'Epoch {epoch:3d} | Loss: {avg_loss:.4f} | Test Acc: {test_acc:.4f}')

print('\n✅ Training complete!')

```

```

In [ ]: # — Visualise training —————
plt.figure(figsize=(8, 5))
plt.plot(train_losses, linewidth=1.5, color='steelblue')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('PyTorch Training Loss')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

5. Regularisation — Preventing Overfitting

Three key techniques:

```

In [ ]: # — Deeper network with regularisation —————

class RegularisedNet(nn.Module):
    """
    Demonstrates three regularisation techniques:
    1. Dropout: randomly zero out neurons during training
    2. Batch Normalisation: normalise layer outputs
    3. Weight Decay: L2 penalty (set in optimiser, not model)
    """
    def __init__(self, n_input, n_hidden, n_output, dropout_rate=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_input, n_hidden),
            nn.BatchNorm1d(n_hidden),      # ← Batch Normalisation
            nn.ReLU(),
            nn.Dropout(dropout_rate),      # ← Dropout: randomly kill 30% c

            nn.Linear(n_hidden, n_hidden // 2),
            nn.BatchNorm1d(n_hidden // 2),
            nn.ReLU(),
            nn.Dropout(dropout_rate),

            nn.Linear(n_hidden // 2, n_output),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.net(x)

reg_model = RegularisedNet(4, 32, 1, dropout_rate=0.3)
print(reg_model)
print(f'\nTotal parameters: {sum(p.numel() for p in reg_model.parameters())}')
print(f'\n💡 Dropout is ONLY active during training. During eval (model.eval())')
print('    Batch Norm normalises each mini-batch to mean=0, std=1. Speeds up

```

6. Optimisers — Beyond Basic Gradient Descent

| Optimiser | Description | When to Use | |----|----|----|----| | **SGD** | Vanilla gradient descent | Simple baselines | | **SGD + momentum** | Adds velocity (like a ball rolling) | CNNs (image tasks) | | **Adam** | Adaptive learning rates per-parameter | **Default for most tasks** | | **AdamW** | Adam + decoupled weight decay | **Default for Transformers/LLMs** |

$$\text{Adam : } m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{momentum})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{adaptive lr})$$

$$W_{t+1} = W_t - \alpha \cdot \frac{m_t}{\sqrt{v_t} + \epsilon}$$

```
In [ ]: # — Compare optimisers —————

def train_with_optimizer(opt_name, opt_class, lr, **kwargs):
    torch.manual_seed(42)
    m = SimpleNet(4, 8, 1)
    opt = opt_class(m.parameters(), lr=lr, **kwargs)
    losses = []
    for epoch in range(200):
        for bx, by in train_loader:
            pred = m(bx)
            loss = loss_fn(pred, by)
            opt.zero_grad()
            loss.backward()
            opt.step()
        losses.append(loss.item())
    return losses

results = {
    'SGD (lr=0.1)': train_with_optimizer('sgd', optim.SGD, 0.1),
    'SGD+Momentum': train_with_optimizer('sgd_m', optim.SGD, 0.1, momentum=0.9),
    'Adam (lr=0.01)': train_with_optimizer('adam', optim.Adam, 0.01),
    'AdamW (lr=0.01)': train_with_optimizer('adamw', optim.AdamW, 0.01, weight_decay=0.01)
}

plt.figure(figsize=(10, 5))
for name, losses in results.items():
    plt.plot(losses, label=name, linewidth=1.5)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Optimiser Comparison')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('💡 Adam/AdamW converge faster and more smoothly.')
print('AdamW is the standard for fine-tuning LLMs (you\'ll see this in No
```

7. Saving & Loading Models

Two approaches:

- `state_dict` : save only weights (recommended)
- `torch.save(model)` : save entire model (fragile, not recommended)

```
In [ ]: # — Save and load —————
import tempfile, os

tmp_dir = tempfile.mkdtemp()
model_path = os.path.join(tmp_dir, 'model.pt')

# Save
torch.save(model.state_dict(), model_path)
print(f'Saved model to {model_path}')
print(f'File size: {os.path.getsize(model_path)} bytes')

# Load into a NEW model instance
new_model = SimpleNet(4, 8, 1)
new_model.load_state_dict(torch.load(model_path, weights_only=True))
new_model.eval() # set to evaluation mode

# Verify same predictions
with torch.no_grad():
    original_pred = model(X_test)
    loaded_pred = new_model(X_test)
    assert torch.allclose(original_pred, loaded_pred)

print('\n✅ Loaded model produces identical predictions.')
print('💡 HuggingFace models use the same pattern under the hood.')
```

✅ Key Takeaways

1. **Tensors** = NumPy arrays + GPU + autograd
2. **Autograd** = automatic backpropagation (no more manual chain rule)
3. **nn.Module** = clean abstraction for building networks
4. **Training loop**: forward → loss → backward → step (memorise this pattern)
5. **Adam/AdamW** = default optimiser for most deep learning
6. **Regularisation**: Dropout + BatchNorm + Weight Decay prevent overfitting
7. **Save with state_dict**, load into a fresh model instance

Next: [05 — NLP & Text Processing](#) — applying deep learning to text data